МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность АС-67

Выполнил
Масюк А. М.,
студент группы АС-67

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«___> _____2026 г.

Брест 2026

Цель: на практике сравнить работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Задачи:

1. Загрузить датасет по варианту;
2. Разделить данные на обучающую и тестовую выборки;
3. Обучить на обучающей выборке три модели: k-NN, Decision Tree и SVM;
4. Для модели k-NN исследовать, как меняется качество при разном количестве соседей (k);
5. Оценить точность каждой модели на тестовой выборке;
6. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Вариант 6

• Pima Indians Diabetes

Предсказать наличие диабета у пациенток

• Задания:

1. Загрузите данные и стандартизируйте их;
2. Разделите выборку в соотношении 70/30;
3. Обучите модели k-NN (с k=5), Decision Tree (с max_depth=4) и SVM (с линейным ядром);
4. Сравните производительность моделей с помощью метрики recall для положительного класса (наличие диабета).
5. Обоснуйте, какой из алгоритмов предпочтительнее для данной медицинской задачи.

Код программы:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score,
    RocCurveDisplay
)
# 1. Загрузка набора данных
columns = [
    "Pregnancies",
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI",
    "DiabetesPedigreeFunction",
    "Age",
```

```
    "Outcome"
]

file_path = r"C:\Users\alex0\OneDrive\Рабочий стол\Лабы3\pima-indians-
diabetes.csv"

try:
    data = pd.read_csv(
        file_path,
        comment="#",
        header=None,
        names=columns
    )
except FileNotFoundError:
    print("Ошибка: файл датасета не найден!")
    input("Нажмите Enter для выхода...")
    raise SystemExit
print("=" * 60)
print("АНАЛИЗ НАБОРА ДАННЫХ PIMA INDIANS DIABETES")
print("=" * 60)
print("\nПервые пять строк:")
print(data.head())
print("\nРазмер набора данных:", data.shape)
# 2. Подготовка данных
# Замена нулевых значений на пропуски
columns_to_replace = [
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI"
]
data[columns_to_replace] = data[columns_to_replace].replace(
    0, np.nan
)
# Разделение признаков и целевой переменной
X = data.drop("Outcome", axis=1)
y = data["Outcome"].astype(int)
print("\nКоличество пропущенных значений:")
print(X.isnull().sum())
print("\nРаспределение классов:")
print(y.value_counts())
# 3. Разделение данных на выборки
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y
)
print("\nОбучающая выборка:", X_train.shape)
print("Тестовая выборка:", X_test.shape)
# 4. Создание моделей

# Модель k-ближайших соседей
knn = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("classifier", KNeighborsClassifier(n_neighbors=5))
])

# Дерево решений
tree = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
```

```
    ("scaler", StandardScaler()),
    ("classifier", DecisionTreeClassifier(
        max_depth=4,
        random_state=42
    ))
])

# Метод опорных векторов
svm = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("classifier", SVC(
        kernel="linear",
        probability=True,
        random_state=42
    ))
])
models = {
    "k-NN": knn,
    "Decision Tree": tree,
    "SVM": svm
}
# 5. Обучение и оценка моделей
results = {}

for name, model in models.items():

    print("\n" + "=" * 60)
    print("МОДЕЛЬ:", name)
    print("=" * 60)

    # Обучение модели
    model.fit(X_train, y_train)

    # Предсказание классов
    predictions = model.predict(X_test)

    # Вероятности положительного класса
    probabilities = model.predict_proba(X_test)[:, 1]

    # Вычисление метрик
    accuracy = accuracy_score(y_test, predictions)

    precision = precision_score(
        y_test, predictions, zero_division=0
    )

    recall = recall_score(
        y_test, predictions, pos_label=1, zero_division=0
    )

    f1 = f1_score(
        y_test, predictions, zero_division=0
    )

    auc = roc_auc_score(y_test, probabilities)

    # Сохранение результатов
    results[name] = {
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-score": f1,
        "ROC AUC": auc
```

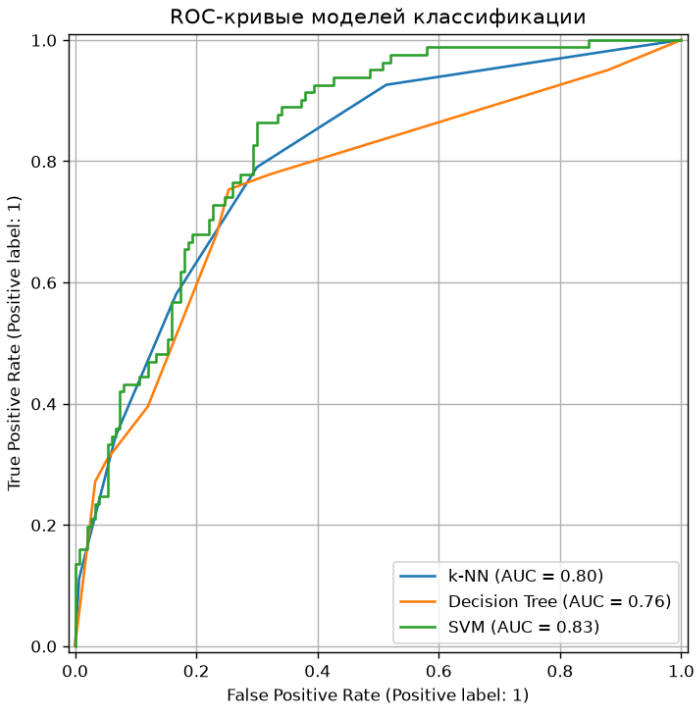
```
    }

    print("\nМетрики качества:")
    print("Accuracy:", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall:", round(recall, 4))
    print("F1-score:", round(f1, 4))
    print("ROC AUC:", round(auc, 4))

    # Матрица ошибок
    print("\nМатрица ошибок:")
    print(confusion_matrix(
        y_test, predictions, labels=[0, 1]
    ))

    # Отчёт классификации
    print("\nОтчёт классификации:")
    print(classification_report(
        y_test,
        predictions,
        labels=[0, 1],
        target_names=["Нет диабета", "Есть диабет"],
        zero_division=0
    ))
# 6. Сравнение моделей
results_df = pd.DataFrame(results).T
print("\n" + "=" * 60)
print("СРАВНЕНИЕ МОДЕЛЕЙ")
print("=" * 60)
print(results_df.round(4).to_string())
# 7. Выбор лучшей модели по Recall
best_model = results_df["Recall"].idxmax()
best_recall = results_df.loc[best_model, "Recall"]
print("\n" + "=" * 60)
print("ЛУЧШАЯ МОДЕЛЬ")
print("=" * 60)
print("Название модели:", best_model)
print("Recall:", round(best_recall, 4))
print(
    "\nЛучшая модель обнаруживает наибольшую долю "
    "положительных случаев диабета."
)
# 8. Построение ROC-кривых
fig, ax = plt.subplots(figsize=(9, 6))
for name, model in models.items():

    RocCurveDisplay.from_estimator(
        model,
        X_test,
        y_test,
        name=name,
        ax=ax
    )
ax.set_title("ROC-кривые моделей классификации")
ax.grid(True)
plt.tight_layout()
plt.show()
# 9. Сохранение результатов
results_df.to_csv(
    "results.csv",
    encoding="utf-8-sig"
)
print("\nРезультаты сохранены в файл results.csv")
input("\nНажмите Enter для завершения программы...")
```




```
Первые пять строк:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  DiabetesPedigreeFunction  Age  Outcome
0            6      148             72             35        0  33.6                     0.627   50        1
1            1       85             66             29        0  26.6                     0.351   31        0
2            8      183             64              0        0  23.3                     0.672   32        1
3            1       89             66             23       94  28.1                     0.167   21        0
4            0      137             40             35      168  43.1                     2.288   33        1

Размер набора данных: (768, 9)

Количество пропущенных значений:
Pregnancies                 0
Glucose                     5
BloodPressure              35
SkinThickness             227
Insulin                   374
BMI                        11
DiabetesPedigreeFunction    0
Age                         0
dtype: int64

Распределение классов:
Outcome
0    500
1    268
Name: count, dtype: int64

Обучающая выборка: (537, 8)
Тестовая выборка: (231, 8)
```

```
============================================================
МОДЕЛЬ: k-NN
============================================================

Метрики качества:
Accuracy: 0.7446
Precision: 0.6528
Recall: 0.5802
F1-score: 0.6144
ROC AUC: 0.8034

Матрица ошибок:
[[125  25]
 [ 34  47]]

Отчёт классификации:
              precision    recall  f1-score   support

 Нет диабета       0.79      0.83      0.81       150
 Есть диабет       0.65      0.58      0.61        81

    accuracy                           0.74       231
   macro avg       0.72      0.71      0.71       231
weighted avg       0.74      0.74      0.74       231
```

```
============================================================
МОДЕЛЬ: Decision Tree
============================================================

Метрики качества:
Accuracy: 0.71
Precision: 0.64
Recall: 0.3951
F1-score: 0.4885
ROC AUC: 0.761

Матрица ошибок:
[[132  18]
 [ 49  32]]

Отчёт классификации:
              precision    recall  f1-score   support

 Нет диабета       0.73      0.88      0.80       150
 Есть диабет       0.64      0.40      0.49        81

    accuracy                           0.71       231
   macro avg       0.68      0.64      0.64       231
weighted avg       0.70      0.71      0.69       231
```

```
Метрики качества:
Accuracy: 0.7273
Precision: 0.6552
Recall: 0.4691
F1-score: 0.5468
ROC AUC: 0.8328

Матрица ошибок:
[[130  20]
 [ 43  38]]

Отчёт классификации:
              precision    recall  f1-score   support

 Нет диабета       0.75      0.87      0.80       150
 Есть диабет       0.66      0.47      0.55        81

    accuracy                           0.73       231
   macro avg       0.70      0.67      0.68       231
weighted avg       0.72      0.73      0.71       231

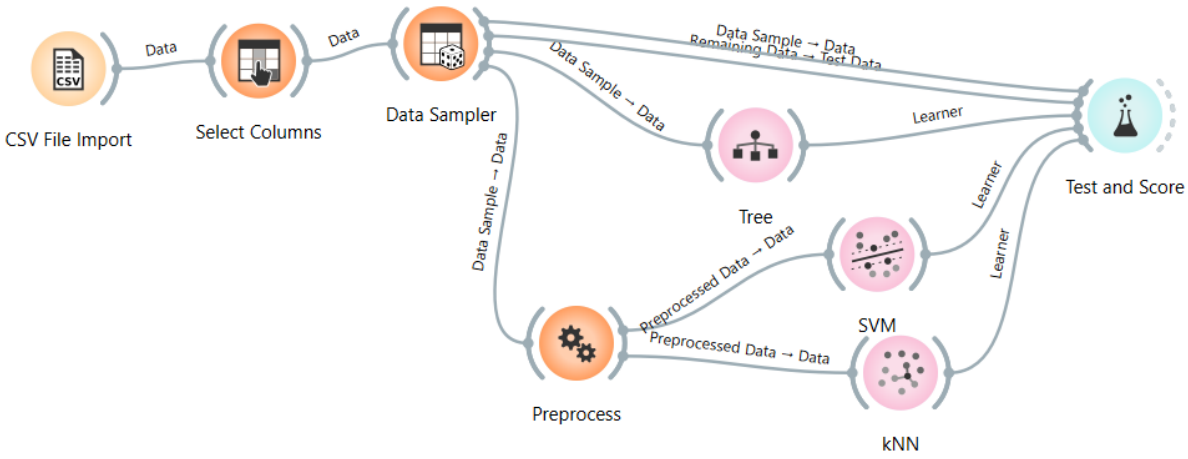

============================================================
СРАВНЕНИЕ МОДЕЛЕЙ
============================================================
               Accuracy  Precision  Recall  F1-score  ROC AUC
k-NN             0.7446     0.6528  0.5802    0.6144   0.8034
Decision Tree    0.7100     0.6400  0.3951    0.4885   0.7610
SVM              0.7273     0.6552  0.4691    0.5468   0.8328

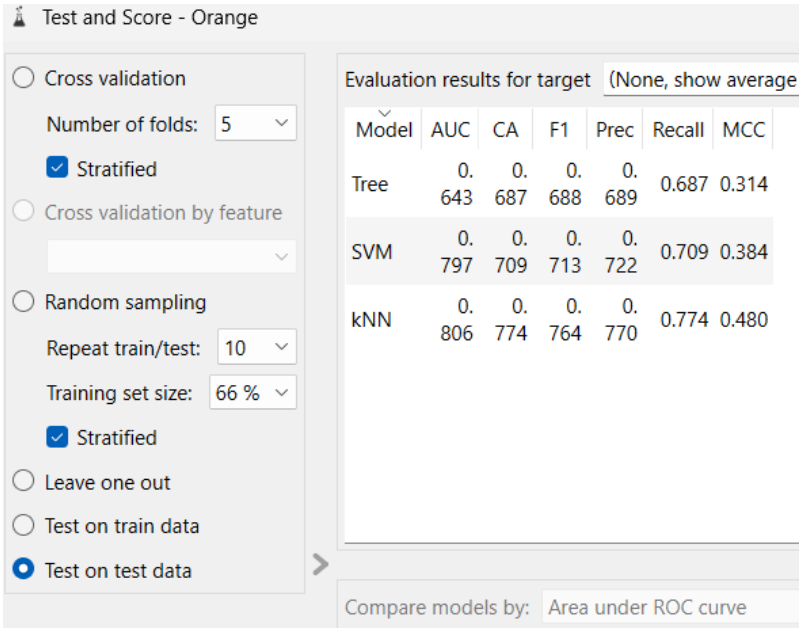
============================================================
ЛУЧШАЯ МОДЕЛЬ
============================================================
Название модели: k-NN
```

```
============================================================
СРАВНЕНИЕ МОДЕЛЕЙ
============================================================
               Accuracy  Precision  Recall  F1-score  ROC AUC
k-NN             0.7446     0.6528  0.5802    0.6144   0.8034
Decision Tree    0.7100     0.6400  0.3951    0.4885   0.7610
SVM              0.7273     0.6552  0.4691    0.5468   0.8328

============================================================
ЛУЧШАЯ МОДЕЛЬ
============================================================
Название модели: k-NN
Recall: 0.5802

Лучшая модель обнаруживает наибольшую долю положительных случаев диабета.
```





Для данной задачи лучше всего подходит алгоритм k-ближайших соседей (k-NN). Он показал лучшие результаты по сравнению с деревом решений и методом опорных векторов. У него самые высокие показатели Accuracy (0,7739), Recall (0,7739) и AUC (0,8058).

Так как задача связана с выявлением сахарного диабета, важно правильно определять пациентов, которые могут иметь это заболевание. Поэтому k-NN можно считать наиболее подходящим из трёх рассмотренных алгоритмов.